

C# (.NET)

Programación orientada a objetos

Archivos de Clases: archivos en los que colocamos las clases. No hay una regla preestablecida en cuanto a organización, se puede colocar cada clase en un archivo, varias clases en uno, una clase en un módulo,.....

Las clases viven en espacios de nombres (NameSpace). Para utilizarlos es necesario importarlos: a través de la instrucción using(a nivel de bloque de archivo)

Las Clases

Una vez abierto un proyecto(del tipo que sea) , elegimos la opción Agregar clase del menú Proyecto, el IDE genera una clase vacía y la guarda en un archivo de código nuevo.

Miembros de una Clase

Existen dos formas de almacenar los datos de una clase: *campos y/o propiedades*. La diferencia entre ambos es muy sutil, en los dos casos almacenamos datos de la clase

- Campos: no tenemos reglas de validación para controlar los valores que les asigna el usuario de mi clase. No tenemos procedimiento de propiedad.
- Propiedades: podemos validar sus valores, así como decidir si queremos que sean de lectura/escritura.
- Métodos: constructores, destructor, funciones

Campos/Propiedades

Se declaran las correspondientes

Class Empleado

[Ambito] [readonly] Identificador as integer

Si el ámbito es público para manipularlos instanciamos un objeto y a través de él manipulamos el campo. Las propiedades son siempre privadas

Si el ámbito es privado tengo que escribir métodos que me permitan acceder al campo (get, set de Java)

Propiedades

Se definen mediante dos elementos: variable y un descriptor de acceso. La variable de propiedad es una variable privada a nivel de clase, y el descriptor es el encargado de realizar las operaciones de acceso y asignación de valores a dicha variable. Consta de dos bloques de código Get para devolver valor y Set para asignar valor

```
Public Class Alumno{  
    private String nom;  
    public string nombre  
        Get{  
            Return nom;  
        }  
  
        Set {  
            nom = value;  
        }  
}
```

Si la propiedad es de lectura no se pondrá bloque set, si la queremos sólo de escritura no se pondrá bloque get

Usando propiedades tenemos la ventaja de escribir código de validación en los correspondientes descriptores.

No es obligatorio que un procedimiento de propiedad esté ligado a una variable de propiedad, a estas se las denomina propiedades virtuales

Definir el comportamiento de la clase. Métodos

Los métodos son los elementos de la clase que definen su comportamiento, se pueden sobrecargar igual que el resto de procedimientos.

Los constructores

Si no se definen C# proporciona el constructor por defecto que inicializa las variables. Pueden sobrecargarse

Herencia

```
Public class Padre{
    .....
}

public class Hija1: Padre{
    .....
}
```

El ámbito igual que java: public, private, protected

El constructor de la clase hija llama al de la clase padre con base(lista de parámetros)

```
public class Hija1(parámetros):base(parámetros del padre)
```

Para llamar a un método de la clase padre se utiliza también este identificador base.método (equivale a super() en java)

Polimorfismo

Si queremos sobrescribir un método de la clase base en una clase hija, tenemos que anteponer la palabra virtual en la base y override en la hija. Si el método no se implementa en la clase base entonces es abstract y la clase también, el identificador override se mantiene en la derivada.

Clases Selladas o no heredables

Si no queremos que una clase nuestra sea heredada por otras, la declaramos con la palabra clave sealed.

Clase Object

Todas las clases heredan de object, gracias a la herencia su funcionalidad también se hereda. Algunos métodos muy utilizados:

```
Public virtual bool equals(object)  
Public Type GetType()
```